

Automation & QA Developer Assessment — Solution & Approach

Senior-engineer analysis · plan of action · all three tasks · defense prep

Task 1 QA report · Task 2 n8n API workflow · Bonus uptime monitor. This document explains the reasoning behind every choice — written for the 30-minute defense call.

Prince Sharma

1. Senior-engineer analysis

This is a 2-hour Automation & QA take-home with three parts and — crucially — a **follow-up call where every choice must be defended** ("why this API, why this node, why this threshold"). So the strategy is not just to produce artifacts, but artifacts I can explain. Three parts:

Part	What it really tests	Key deliverables
Task 1 — QA report	Can I find real bugs in a vibe-coded app and explain them to a non-technical stakeholder?	Bug table (5+ issues) + one root-cause analysis
Task 2 — n8n workflow	Can I wire 2 APIs with transformation, a conditional branch, output, and real error handling — no hard-coded secrets?	Workflow JSON + 2 screenshots + README
Bonus — uptime monitor	Combine both skills. "Strong submissions almost always attempt this."	Workflow JSON + screenshot

> **Reality check:** this assessment requires *running* things — installing n8n, exercising a live app, building on a canvas, capturing **your own** executions, and recording a Loom of **you** explaining it. The artifacts here are fully prepared and importable, but the screenshots, bug confirmations, and video must come from your real runs. Every bug you submit must be one you personally reproduced — you defend them in the call.

2. Plan of action

1. **Task 2 first (highest-value, most concrete).** Build the importable n8n workflow with all required elements, then verify it posts to Discord.
2. **Bonus next (cheap win, expected).** The uptime monitor reuses the same Discord credential — small effort, big signal.
3. **Task 1 QA.** Exercise the RealWorld demo, confirm the candidate bugs, drop any that don't reproduce, keep 5+.
4. **Package & record.** Public GitHub repo, 1-page README, Loom walkthrough.

3. Task 1 — QA report (summary)

App tested: **RealWorld "Conduit" demo**. I exercised all main flows plus edge cases and documented **7 candidate issues** (so you can drop any that don't reproduce and still submit five). Severity-ranked, with repro steps and suspected cause. The root-cause analysis covers the **stale-authentication** issue: the app trusts a local-storage token without re-validating it and has no global 401 handler, so expired sessions fail silently — fixed with a 401 interceptor that logs the user out plus token validation on load.

Full report: `Task1_QA_Report_PrinceSharma.pdf`.

4. Task 2 — n8n workflow design



Requirement	How it's met
Trigger (schedule OR webhook)	Both: a Schedule trigger (hourly) and a Webhook trigger for manual curl testing
First HTTP request	GitHub repo search by topic, sorted by stars
Transformation	Code node keeps the top 5 and reshapes to needed fields
Second HTTP request (enrich)	Fetches each repo's README metadata (size)
Conditional branch	IF node: stars > threshold → HOT, else STANDARD (threshold is config-driven)
Output	Formatted digest posted to Discord
Error handling	Continue-On-Fail + retries on every HTTP node; failures routed to a dedicated error node that logs a safe summary — never silent
No hard-coded secrets	Discord webhook stored in n8n Credentials, referenced by the node

5. Bonus — uptime monitor design

Pings the Task-1 app **every 5 minutes**, reads the HTTP status with `neverError` (so even a 5xx is inspected, not thrown), and an IF node posts a Discord **alert on any non-200**. Extras beyond the brief: **3× retry** (so a single blip doesn't page anyone), and **response-time tracking** in the alert.

6. Defense prep — expected follow-up questions

The brief states you must defend every choice. These are the likely questions and crisp answers:

Q: Why the GitHub API? Free, no key needed for basic rate limits, returns clean structured JSON, and gives a naturally defensible threshold (stars). Easy to reason about live.

Q: Why the README enrichment as the second call? It's a real second request on real data and demonstrates chaining a per-item follow-up — the pattern the task is testing — rather than a throwaway call.

Q: Why a threshold of 20,000 stars? It cleanly separates standout repos from the long tail for the chosen topic. It's config-driven (in the Set Config node), so it's trivially tunable per topic in production — I wouldn't bake a magic number into the IF node.

Q: Why Discord for output? Fastest free notification channel — a webhook URL with no OAuth or app review. The output node is swappable for Slack/Telegram/Sheets with one change.

Q: How does your error handling actually work? Every HTTP node uses Continue-On-Fail with retries, so transient failures self-heal. A genuine failure routes to a dedicated error node that logs a structured, non-sensitive summary (node, message, timestamp) — so a failed call is logged, never silent, and never crashes the run.

Q: Why not hard-code the webhook URL? Secrets belong in n8n's Credentials store — rotatable, not committed to Git, and not exposed in the exported JSON. Hard-coding would leak the URL in the public repo.

Q: Why retry in the uptime monitor? A single failed ping is often a transient network blip, not an outage. Three retries before alerting prevents false alarms while still catching real downtime within the 5-minute window.

7. FAQ compliance (from the assessment)

- **AI tools allowed?** Yes — used to scaffold, but every decision above is one I can defend in the call.
- **Prettify the workflow?** No — working over pretty. Time went into error handling and the bonus instead, exactly as advised. Canvases are laid out in execution order for readability, not decoration.
- **Demo app has no obvious bugs?** Covered — the report uses edge-case load (long/special input, throttling, expired session) to surface real issues, and lists 7 candidates.
- **Tests?** Not required; the webhook can be smoke-tested with one curl (documented in the README).

8. Submission checklist

- [ ] Public GitHub repo with both workflow JSONs + READMEs + the QA report PDF
- [ ] Task 1: 5+ personally-verified bugs with screenshots
- [ ] Task 2: workflow screenshots (canvas + successful execution)
- [ ] Bonus: workflow screenshot + a forced-failure alert screenshot
- [ ] Loom video (use the script in `LOOM_SCRIPT_AND_VERIFY.md`)
- [ ] 1-page documentation (the repo README covers this)